

Review Intelligence

An End-to-End NLP Project: from pretrained transformers to a fine-tuned, deployed model

Rahiq Raees

July 2026

[GitHub Repository](#) | [Live Demo](#) | [Fine-tuned Model](#)

Contents

1	Executive Summary	1
2	Motivation and Objective	1
3	What the System Does	2
4	Technical Approach	2
4.1	Stage 1 — Baseline with pretrained pipelines	2
4.2	Stage 2 — Understanding the internals	2
4.3	Stage 3 — Fine-tuning a domain-specific model	3
4.4	Stage 4 — Publishing and deployment	3
5	Results	3
6	Software Engineering Practices	3
7	Skills and Tools Demonstrated	4
8	Limitations and Future Work	4
9	Links and Reproducibility	4

1 Executive Summary

Review Intelligence is an end-to-end natural language processing (NLP) system that turns unstructured customer reviews into structured, actionable signal: **sentiment**, **topics**, and a short **summary**.

The project was built deliberately as one evolving system rather than a set of disconnected tutorials. It progresses from off-the-shelf transformer models, through a deep understanding of how those models work internally, to a **custom model fine-tuned on real product-review data**, and finally to a **live, publicly hosted web application** that serves that model.

Key outcomes:

- Fine-tuned a DistilBERT transformer that improved product-review sentiment accuracy from **88.6% to 89.7%** (F1 0.899) over a strong pretrained baseline, with an honest, controlled evaluation.

- Published the fine-tuned model to the Hugging Face Hub and deployed an interactive demo on Hugging Face Spaces that serves it live.
- Engineered the codebase to a production standard: an installable Python package, an automated test suite, and continuous integration (CI).

2 Motivation and Objective

Businesses receive customer feedback as free-form text at a volume no team can read manually. Automatically classifying each review’s sentiment, identifying which topics it touches (shipping, product quality, price, support), and summarizing it are the building blocks of a feedback dashboard that tells a company *what* customers are unhappy about and *how much*.

The objective was to build such a system while demonstrating the full modern NLP workflow — using pretrained transformer models, understanding their internals, adapting them to a specific domain through fine-tuning, and deploying the result — all with the software-engineering rigor expected in production (reproducible environments, testing, CI, and deployment).

3 What the System Does

The system exposes three analyzers, each deliberately built on a different transformer *architecture*. This mapping is a core NLP concept: different model designs suit different tasks.

Capability	Task	Transformer architecture
Sentiment scoring	Text classification	Encoder (DistilBERT) — reads the whole sentence to <i>understand</i> it
Topic detection	Zero-shot classification	Encoder (BART fine-tuned on natural language inference)
Summarization	Sequence-to-sequence	Encoder–decoder (DistilBART) — reads the review and <i>generates</i> a shorter one

Example. Given the review “*The product itself is fine, but it took three weeks to arrive and the box was crushed,*” the system returns sentiment **NEGATIVE**, top topic *shipping and delivery*, and a one-line summary.

4 Technical Approach

The system was built in four stages, each adding a capability and a deeper level of understanding.

4.1 Stage 1 — Baseline with pretrained pipelines

The first version used Hugging Face `pipeline()` objects — a high-level interface to run production-grade transformer models with a single call. This established a working baseline on real reviews and demonstrated the three transformer families (encoder, decoder, encoder–decoder).

A notable technique here is **zero-shot classification** for topic detection: rather than training a classifier on labeled topic data, the system reframes “is this review about shipping?” as a natural-

language-inference question. This lets the topic labels be defined at run time and changed freely without any retraining.

4.2 Stage 2 — Understanding the internals

To move beyond treating the models as black boxes, the sentiment pipeline was reproduced manually, stage by stage:

- **Tokenizer:** converts text into integer token IDs and adds the special tokens the model expects, producing an “attention mask” that marks real tokens versus padding.
- **Model:** runs the transformer to produce raw scores (logits).
- **Post-processing:** applies a softmax to convert logits into probabilities, then selects the highest-scoring label.

The manual result matched the pipeline’s output exactly, confirming a correct understanding of every step — a prerequisite for the fine-tuning that follows.

4.3 Stage 3 — Fine-tuning a domain-specific model

The baseline sentiment model is trained on *movie* reviews (the SST-2 dataset), while this project analyzes *product* reviews — a different domain with different vocabulary. To close this gap, a DistilBERT model was **fine-tuned** (an application of transfer learning) on a streamed, class-balanced subset of the Amazon product-reviews dataset.

Methodology:

- Data loaded and processed with the Hugging Face `datasets` library; sequences tokenized in batches with dynamic padding for efficiency.
- Training run with the Hugging Face `Trainer` API (6,000 training reviews, 2 epochs, learning rate 2×10^{-5}).
- Evaluated honestly: the fine-tuned model and the movie-trained baseline were both scored on the *same* 1,000 held-out product reviews, using accuracy and F1.

4.4 Stage 4 — Publishing and deployment

The fine-tuned model was published to the Hugging Face Hub as a model repository with a model card documenting its data, results, and usage. An interactive web application was built with Gradio and deployed to Hugging Face Spaces, where it runs on hosted CPU hardware and serves the fine-tuned model to any user with the link.

5 Results

Model (evaluated on product reviews)	Accuracy	F1
Baseline (movie-trained, SST-2)	88.6%	0.888
Fine-tuned (product reviews)	89.7%	0.899

Interpretation. The improvement of +1.1 percentage points is real but modest, and understanding *why* is part of the result. Binary sentiment transfers well across domains — “I love it” reads the same in a movie or a product review — so the off-the-shelf baseline was already strong, leaving a small domain gap to close. Reporting a genuine, well-measured improvement against a strong baseline, and understanding its size, reflects sound evaluation practice: a controlled comparison on a held-out set, rather than an inflated number from a weak baseline or a leaky test.

6 Software Engineering Practices

The project was engineered like production software, not a one-off notebook:

- **Installable package** with a clean, reusable API separating library code from runnable scripts.
- **Reproducible environment** pinned via a Conda environment file, including a deliberate version pin to keep results stable across library updates.
- **Automated tests** (pytest), split into fast unit tests (no model downloads, safe for CI) and slower integration tests.
- **Continuous integration** via GitHub Actions, running the test suite across Python 3.10 and 3.11 on every change.
- **Deployment** of both a model artifact (to the Hub) and a live application (to Spaces).

7 Skills and Tools Demonstrated

- **NLP / Transformers:** tokenization, model internals, encoder vs. encoder-decoder architectures, zero-shot classification, fine-tuning, transfer learning.
- **Ecosystem:** Hugging Face Transformers, Datasets, Hub, and Spaces; PyTorch; Gradio.
- **Evaluation:** controlled baseline comparison, accuracy and F1, train/test hygiene.
- **Engineering:** Python packaging, pytest, GitHub Actions CI, reproducible environments, model and application deployment.

8 Limitations and Future Work

- **Binary sentiment.** The current model predicts positive or negative only; genuinely mixed reviews are forced into one class. A natural next step is a three-class (negative / neutral / positive) model to capture nuance.
- **Demo hardware.** The live demo runs on free CPU hardware, so the first request is slow while models load, then fast.
- **Next project.** The same skills are being extended to a retrieval-augmented question-answering (RAG) system in a new domain, combining semantic search with a question-answering model.

9 Links and Reproducibility

- **Source code:** github.com/rahiqraees/review-intelligence

- **Live demo:** huggingface.co/spaces/rahiqr/review-intelligence
- **Fine-tuned model:** huggingface.co/rahiqr/distilbert-amazon-sentiment

The repository includes setup instructions, runnable scripts for each stage, and the recorded evaluation metrics, so the full pipeline can be reproduced from scratch.